

Sistemas Compostos de IA: Otimização de Respostas em Benchmarks Objetivos por meio de Orquestração Baseada no Ralph Loop

Hugo Emílio Nomura, Pedro Henrique Correia de Oliveira, Pedro Henrique Satoru Lima Takahashi, Vitor Monteiro Vianna

Ciência da Computação – Centro Universitário FEI, campus São Paulo

unifhnomura@fei.edu.br, unifpoliveira@fei.edu.br, unifptakahashi@fei.edu.br, unievvianna@fei.edu.br

Orientador: Charles Henrique Porto Ferreira

Departamento de Ciência da Computação, Centro Universitário FEI

cferreira@fei.edu.br

Resumo: Embora os *Large Language Models* (LLMs) continuem operando como geradores de texto, sua evolução recente não depende apenas do aumento de capacidade ou da complexidade matemática. Soluções modernas incorporam planejamento, refinamento e ciclos iterativos de raciocínio para melhorar a qualidade e a consistência das saídas. No entanto, o uso prolongado desses mecanismos amplia a janela de contexto e pode causar degradação de informação (*context rot*), reduzindo a confiabilidade das respostas.

Este trabalho propõe adaptar os princípios do Ralph Wiggum Loop para cenários gerais de *benchmarking*, com o objetivo de melhorar respostas por meio de chamadas iterativas ao próprio modelo, sem ferramentas externas. Para mitigar o *context rot*, a arquitetura preserva um comportamento *stateless* entre as fases, mantendo apenas os artefatos estritamente necessários a cada etapa.

O impacto empírico da abordagem será avaliado com os *benchmarks* MMLU e GPQA, comparando a acurácia de chamadas diretas ao modelo com a de execuções mediadas pelo *harness* proposto.

Palavras-chaves: *Large Language Models* (LLMs). Ralph Wiggum Loop. Sistemas Compostos de IA. Engenharia de Prompt. Decaimento de Contexto (*context rot*).

1. Introdução

Os *Large Language Models* (LLMs) consolidaram-se como o núcleo de uma nova geração de sistemas capazes de executar tarefas complexas de processamento de linguagem natural, incluindo geração de texto, resolução de problemas, planejamento intermediário e suporte à tomada de decisão. Apesar dessas capacidades, os modelos que servem como base para esses sistemas continuam sendo, em essência, geradores probabilísticos de *tokens*, operando a partir de uma entrada de texto e retornando como saída outro texto. Nesse contexto, parte significativa da evolução recente dos sistemas baseados em LLMs passou a ocorrer não apenas nos próprios modelos, mas também nas estruturas responsáveis por organizar sua utilização prática. Trabalhos recentes, como [1], afirmam que “*The performance of large language model (LLM) systems depends not only on model weights, but also on their harness: the code that determines what information to store, retrieve, and present to the model*”, evidenciando o papel dos chamados *harnesses* na ampliação das capacidades desses sistemas. Ferramentas modernas, como o *ChatGPT*, incorporam tais mecanismos para adicionar camadas de coordenação, decomposição de tarefas, validação intermediária e orquestração de múltiplas inferências, permitindo que o modelo execute fluxos significativamente mais sofisticados do que uma única chamada linear de entrada e saída.

Abordagens baseadas em uma única chamada ao modelo, nas quais toda a resposta é produzida em uma única

etapa de inferência, apresentam limitações importantes em tarefas que exigem raciocínio complexo, múltiplas etapas de decisão ou validações intermediárias. Como resposta a esse problema, diferentes linhas de pesquisa passaram a explorar fluxos estruturados de *thinking*, nos quais o processo de inferência é explicitamente decomposto em etapas sucessivas de raciocínio, planejamento, execução e refinamento. Trabalhos baseados em *Chain-of-Thought* [2], *Tree of Thoughts* [3] e na técnica de *Least-to-Most* [4] demonstram que a decomposição explícita do raciocínio favorece significativamente a resolução de tarefas complexas. De maneira semelhante, abordagens como *ReAct* [5] combinam raciocínio e ações iterativas, enquanto métodos como *Reflexion* [6], *Self-Debugging* [7] e *Self-Refine* [8] exploram ciclos sucessivos de avaliação, crítica e refinamento da própria resposta gerada. Em conjunto, esses trabalhos indicam que ganhos de desempenho não dependem exclusivamente do tamanho do modelo ou da quantidade de *tokens* gerados, mas também da maneira como o processo de inferência é estruturado, coordenado e validado ao longo da execução.

Apesar dos ganhos observados nessas arquiteturas iterativas compostas por agentes, ciclos de reflexão ou *pipelines* sucessivos, tais abordagens também introduzem novos desafios relacionados ao gerenciamento do contexto acumulado durante a inferência. Em muitos desses sistemas, o histórico completo de interações permanece armazenado na janela de contexto ao longo de toda a execução como mecanismo de preservação da continuidade do raciocínio.

Embora essa estratégia favoreça rastreabilidade e coerência entre etapas intermediárias, ela também aumenta progressivamente a dificuldade de recuperação de informações relevantes conforme o contexto cresce. Liu et al. [9], por exemplo, demonstram que ocorre perda de desempenho em tarefas que exigem localizar informações relevantes em sequências extensas. Trabalhos recentes também passaram a discutir fenômenos associados à degradação progressiva do contexto em execuções longas, incluindo compressão implícita de informações, perda de relevância contextual e aumento de inconsistências ao longo da inferência. Esses efeitos vêm sendo frequentemente associados, ao conceito de *context rot*, termo utilizado por Hong et al. [10] para descrever a degradação gradual da qualidade das respostas conforme o volume de contexto acumulado aumenta.

Além do avanço acadêmico dessas abordagens, a indústria de engenharia de software também passou a investir intensamente em arquiteturas de *deep thinking* e em *harnesses* especializados para ampliar a capacidade operacional de LLMs em tarefas complexas de desenvolvimento. Ferramentas recentes, como *Claude Code*, *GPT Codex*, *OpenCLI* e *Devin*, exemplificam essa tendência ao combinar planejamento, decomposição de tarefas, validação iterativa e execução coordenada de múltiplas inferências. Nesse contexto, uma das abordagens que ganhou notoriedade foi o *Ralph Wiggum Loop* [11], popularizado por Geoffrey Huntley no contexto de engenharia de software. Em sua formulação original, entretanto, o modelo não foi concebido para cenários generalistas, mas especificamente para fluxos de desenvolvimento baseados em conceitos próprios da engenharia de software, como definição de funcionalidades, requisitos, critérios de aceite e integração com sistemas de versionamento, como o *Git*. Ainda assim, sua estrutura apresenta dois princípios centrais relevantes para os problemas discutidos na literatura de LLMs: a decomposição do processo em tarefas independentes, reduzindo a necessidade de reaproveitamento contínuo do contexto entre etapas, e a execução iterativa de cada tarefa até que critérios mínimos de qualidade sejam satisfatoriamente atingidos.

Este trabalho parte de três premissas consolidadas na literatura: que abordagens de *thinking* podem melhorar a qualidade das respostas de LLMs em cenários específicos, como os de tarefas complexas; que tais abordagens introduzem problemas associados ao crescimento excessivo do contexto acumulado, fenômeno descrito como *context rot*; e que o *Ralph Wiggum Loop* [11], concebido por Huntley para fluxos de engenharia de software, oferece princípios estruturais capazes de mitigar o problema de *context rot* por meio de decomposição de tarefas e isolamento contextual entre etapas. A partir dessas premissas, propõe-se a adaptação do *Ralph Wiggum Loop* para um contexto generalista de resolução de tarefas em linguagem natural a partir de um *prompt* de entrada fornecido pelo usuário. Diferentemente da proposta original, centrada em desenvolvimento de software, definições de funcionalidades, critérios de aceite baseados em requisitos funcionais e integração com ferramentas de versionamento (como *Git*), a adaptação proposta investiga se, ao adaptar a estrutura do *Ralph Wiggum*

Loop a um fluxo completo de *thinking* aplicável à resolução de perguntas gerais e à exploração de problemas diversos e de diferentes complexidades, também haverá melhora na qualidade dessas respostas quando comparadas a uma única etapa de inferência.

Para avaliar essa hipótese de maneira objetiva, reproduzível e quantitativa, o trabalho compara diretamente duas estratégias de inferência: (i) uma chamada tradicional a um modelo de LLM, na qual toda a resposta é produzida em uma única etapa de inferência; e (ii) uma execução mediada pelo *harness* proposto, que adapta o *Ralph Wiggum Loop* para cenários generalistas. O objetivo experimental é medir em que magnitude a aplicação desse fluxo estruturado de *thinking* melhora a qualidade das respostas em relação à inferência singular tradicional. Para permitir essa quantificação, os experimentos utilizam *benchmarks* compostos por questões e respostas previamente definidas, estratégia amplamente adotada para mensuração de ganhos em arquiteturas de raciocínio e orquestração de LLMs, e usada em trabalhos como *ReAct* [5] e *Beyond the Strongest LLM* [12]. Os experimentos são conduzidos com diferentes modelos da família Llama 3.1, com diferentes quantidades de parâmetros utilizados na execução do modelo, representando distintos níveis de complexidade computacional, e avaliados por meio dos *benchmarks* MMLU [13] e GPQA [14], sendo a acurácia, medida pela quantidade de acertos entre a alternativa selecionada pelo sistema de LLM e o gabarito oficial.

II. Referencial Bibliografico

Esta seção apresenta os conceitos fundamentais necessários para a compreensão do trabalho proposto. São definidos os termos técnicos centrais utilizados ao longo do texto, descritas as principais estratégias de raciocínio e refinamento aplicadas a modelos de linguagem, e introduzido o paradigma de orquestração adotado neste trabalho.

A. Modelos de Linguagem e a Arquitetura de Harness

Embora os *Large Language Models* (LLMs) [15] sejam amplamente consolidados como geradores probabilísticos de *tokens* baseados na arquitetura *Transformer* [16], a evolução recente na aplicação dessas tecnologias migrou de chamadas diretas e lineares para o uso de estruturas de orquestração mais robustas. Um detalhamento teórico e histórico aprofundado sobre o funcionamento desses modelos de linguagem pode ser encontrado no levantamento conduzido por Zhao et al. [17].

Na prática, a organização dessas interações é viabilizada por meio de um *harness*, termo que se refere à estrutura lógica e às regras de orquestração que envolvem um modelo de linguagem para guiar seu processo de inferência, funcionando como uma camada de coordenação entre o *prompt* inicial do usuário e a saída do modelo [12], [18]. Um *harness* pode incluir etapas de planejamento, decomposição de tarefas, validação intermediária e composição de respostas finais. Variações na arquitetura do *harness* podem causar impactos significativos no desempenho do sistema como um todo, mesmo quando o modelo base permanece o

mesmo.

Esse conceito está diretamente relacionado à noção de *Sistemas Compostos de IA*, definida por Zaharia et al. [18] como arquiteturas que resolvem tarefas de inteligência artificial por meio de múltiplos componentes que interagem entre si, incluindo chamadas a modelos, recuperadores de informação e ferramentas externas. Segundo essa perspectiva, os ganhos mais significativos em aplicações de IA contemporâneas passaram a depender da engenharia de sistemas que combinam componentes especializados em fluxos coordenados, em vez de se apoiarem unicamente no aumento de escala dos modelos base [18].

Essa tendência se manifesta em ferramentas industriais que utilizam LLMs integrados a *harnesses* especializados para tarefas de engenharia e automação. Exemplos incluem o Claude Code [19], interface de linha de comando voltada ao desenvolvimento assistido por IA; o GPT Codex [20], otimizado para geração e compreensão de código; e o OpenCLI [21], voltado à automação de comandos de terminal. Essas ferramentas demonstram como a combinação de planejamento, decomposição de tarefas e validação iterativa expande a capacidade computacional útil dos modelos base.

B. Estratégias de Raciocínio e Refinamento

A literatura demonstra que a qualidade da inferência de um LLM pode ser melhorada de forma significativa quando o processo é estruturado em fases [2], [3]. O método *Chain-of-Thought* [2] e sua generalização *Tree of Thoughts* [3] mostraram a importância de induzir o modelo a explicitar passos intermediários de raciocínio. Abordagens como *ReAct* [5] ampliaram esse princípio ao combinar o raciocínio interno do modelo com a capacidade de invocar ferramentas externas, permitindo corrigir alucinações por meio de observações diretas do ambiente.

De forma complementar, pipelines baseados em ciclos de *feedback* iterativo também apresentam ganhos consistentes. O framework *Reflexion* [6] introduziu ciclos nos quais o modelo atua como seu próprio crítico, revisando saídas com base em reforço verbal externo. A abordagem de *Self-Debugging* [7] demonstrou que LLMs podem identificar e corrigir seus próprios erros quando guiados de forma sistemática. Na mesma direção, o método *Self-Refine* [8] consolidou a eficácia do refinamento iterativo ao demonstrar que modelos podem gerar *feedback* crítico sobre suas próprias respostas e utilizá-lo para melhorar as saídas de forma sucessiva, contornando limitações da geração em uma única etapa. Essa capacidade de autocrítica e revisão contínua constitui uma premissa fundamental para a proposta deste trabalho.

Além dessas abordagens, métodos focados na decomposição de tarefas complexas também contribuem para a redução de erros lógicos e omissões. Técnicas de *prompting* baseadas em decomposição, como *Least-to-Most* [4], *Decomposed Prompting* [22] e *Plan-and-Solve* [23], sustentam que separar o planejamento da execução e dividir problemas em subproblemas menores melhora o desempenho do modelo, especialmente quando combinado com mecanismos de validação intermediária.

C. Degradação de Contexto

Embora as técnicas apresentadas nas subseções anteriores tenham demonstrado eficácia individual, muitas delas dependem do acúmulo contínuo de estado em uma única janela de contexto, o que pode agravar problemas de degradação à medida que o histórico cresce. Liu et al. [9] demonstraram empiricamente que LLMs apresentam perda significativa de desempenho quando precisam recuperar informações relevantes posicionadas no meio de sequências longas, fenômeno descrito como “perdidos no meio” (*lost in the middle*).

Na prática, esse comportamento se manifesta de diversas formas: compressão implícita de informações, perda de relevância contextual, aumento de inconsistências e maior incidência de alucinações em execuções prolongadas [9], [17]. A literatura contemporânea frequentemente associa esse conjunto de efeitos ao conceito de *context rot*, referindo-se à degradação progressiva da qualidade das respostas conforme o contexto acumula ruído, redundâncias e informações que competem pela atenção do modelo [9].

D. O Ralph Wiggum Loop

Diante dos problemas de degradação de contexto, o *Ralph Wiggum Loop* [11] surgiu na comunidade de engenharia de software como um paradigma de orquestração que aborda diretamente essa limitação. Idealizado por Geoffrey Huntley, o paradigma parte de um conjunto de requisitos fornecidos como entrada e os decompõe em histórias de usuário, conceito consolidado na disciplina de engenharia de software. Cada história é tratada de forma atômica: o modelo implementa uma solução, executa testes e verifica se os critérios de aceite foram satisfeitos; caso contrário, o ciclo se repete até que a história seja concluída com êxito. Ao término de cada história, seus resultados não são carregados diretamente para a execução seguinte, evitando o acúmulo progressivo de estado que caracteriza arquiteturas tradicionais de agentes.

Um aspecto central dessa abordagem é o princípio de *Fresh Context* [11]: ao iniciar cada nova história, o contexto de execução é completamente reinicializado. Antes de qualquer instrução sobre a nova história, os aprendizados gerais extraídos das histórias anteriores são injetados como primeira entrada nesse novo contexto, de forma estruturada e explícita. Dessa maneira, o paradigma preserva o conhecimento relevante acumulado sem herdar o ruído, as redundâncias e as informações obsoletas que degradam progressivamente a qualidade das respostas em execuções longas, mitigando ativamente o fenômeno de *context rot* [10].

Em sua formulação original, o *Ralph Wiggum Loop* foi concebido especificamente para fluxos de engenharia de software, com etapas voltadas à definição de funcionalidades, requisitos, critérios de aceite e integração com sistemas de versionamento como *Git*. Ainda assim, seus princípios de execução *stateless* e validação iterativa apresentam relevância direta para os problemas discutidos na literatura de LLMs. A validação acadêmica dessa abordagem foi reforçada pelo estudo de McComb et al. [24], que avaliou

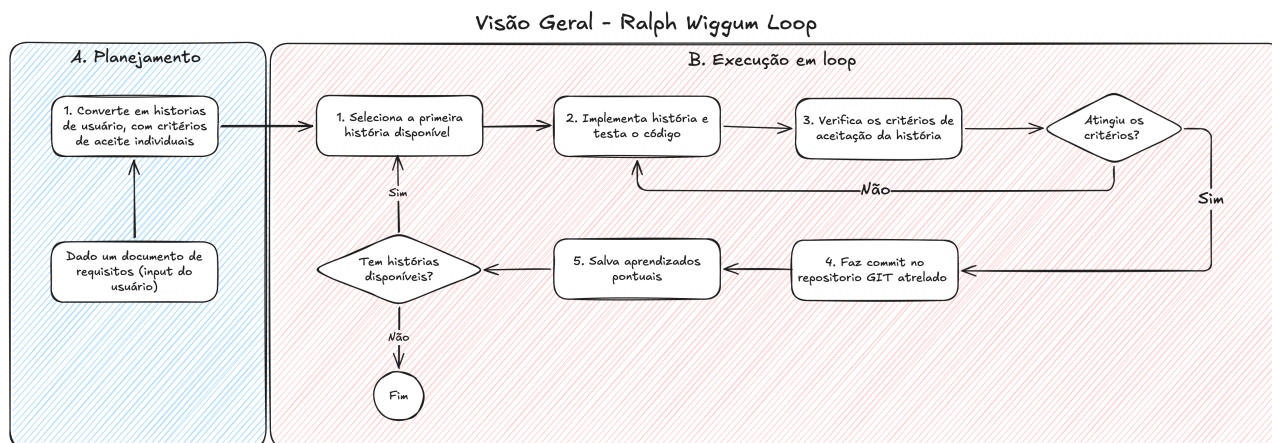


Figura 1. Ralph Wiggum Loop original.

o impacto de agentes baseados no *Ralph Wiggum Loop* na resolução de problemas complexos de design de engenharia, confirmando a eficácia do paradigma *stateless* em contextos que exigem raciocínio estruturado e iterativo. Este trabalho se apoia nesses fundamentos para justificar o uso do paradigma, adaptando sua orquestração para a resolução de questões de propósito geral em múltiplos domínios do conhecimento.

III. Metodologia

A adaptação do *Ralph Wiggum Loop* para um contexto generalista de resolução de perguntas em linguagem natural exige ajustes estruturais em relação à formulação original, que foi concebida exclusivamente para fluxos de engenharia de software. A Figura 2 apresenta uma visão geral do fluxo proposto, que preserva os princípios centrais do paradigma original, representado na Figura 1.

Em linhas gerais, propomos uma conversão do conceito de histórias de usuário para tarefas, o que nos permite manter um fluxo muito próximo do original, contendo as etapas de “Planejamento” e “Execução em Loop”. Entretanto, passa a ser necessário considerar a criação de um retorno ao usuário, algo que não constitui uma preocupação no *Ralph Wiggum Loop* original. Dessa forma, incorporamos uma nova fase após o fluxo de execução, denominada “Criação da Resposta Final”, que contém toda a lógica necessária para agrupar os resultados produzidos pelas tarefas anteriores em uma resposta que apresente o conteúdo e o formato esperados pelo usuário.

Para ilustrar concretamente cada etapa do fluxo, adota-se ao longo desta seção o seguinte *input* de usuário como referência:

Exemplo

Qual dos seguintes números é simultaneamente primo, par e maior que 2?

- A) 2
- B) 4
- C) 6

D) Nenhuma das anteriores

A. Planejamento

O planejamento é o início de todo o fluxo, compreendendo duas etapas que preparam toda a estrutura de execução: a definição dos critérios que a resposta final deverá respeitar e a decomposição da pergunta em tarefas autônomas e verificáveis.

A.1 Definição dos critérios de aceite gerais

Essa etapa representa uma adição em relação ao paradigma original. No contexto de engenharia de software, cada tarefa concluída resultava em um *commit* que preservava o progresso diretamente no repositório, de modo que, ao término de todas as histórias de usuário, o produto final já estava pronto. No fluxo adaptado, entretanto, a conclusão de todas as tarefas não produz automaticamente uma saída utilizável: após todas as tarefas serem concluídas, é necessário construir uma resposta final que consolide os resultados gerados. Os critérios de aceite gerais definem as regras que essa resposta deverá respeitar, abrangendo aspectos como o idioma a ser utilizado, o formato de apresentação esperado e os tópicos-chave que devem constar no conteúdo final.

A implementação dessa etapa utiliza um *template* fixo (Template A.1), no qual o *input* fornecido pelo usuário é injetado. Esse *template* contém um *prompt* que solicita ao modelo a definição de uma lista de critérios de aceite, os quais serão resgatados posteriormente na fase de Criação da Resposta Final. Especificamente, o *template* solicita a definição do idioma a ser usado na resposta final, do formato em que a resposta deve ser construída e dos tópicos-chave do conteúdo que devem estar presentes. A relação de entrada e saída dessa etapa pode ser expressa como:

$$\text{extPerguntadousurio} + \text{Template A.1} \rightarrow \text{Resultado Fase A.1} \quad (1)$$

O resultado produzido é armazenado localmente em arquivo ou na memória do programa.

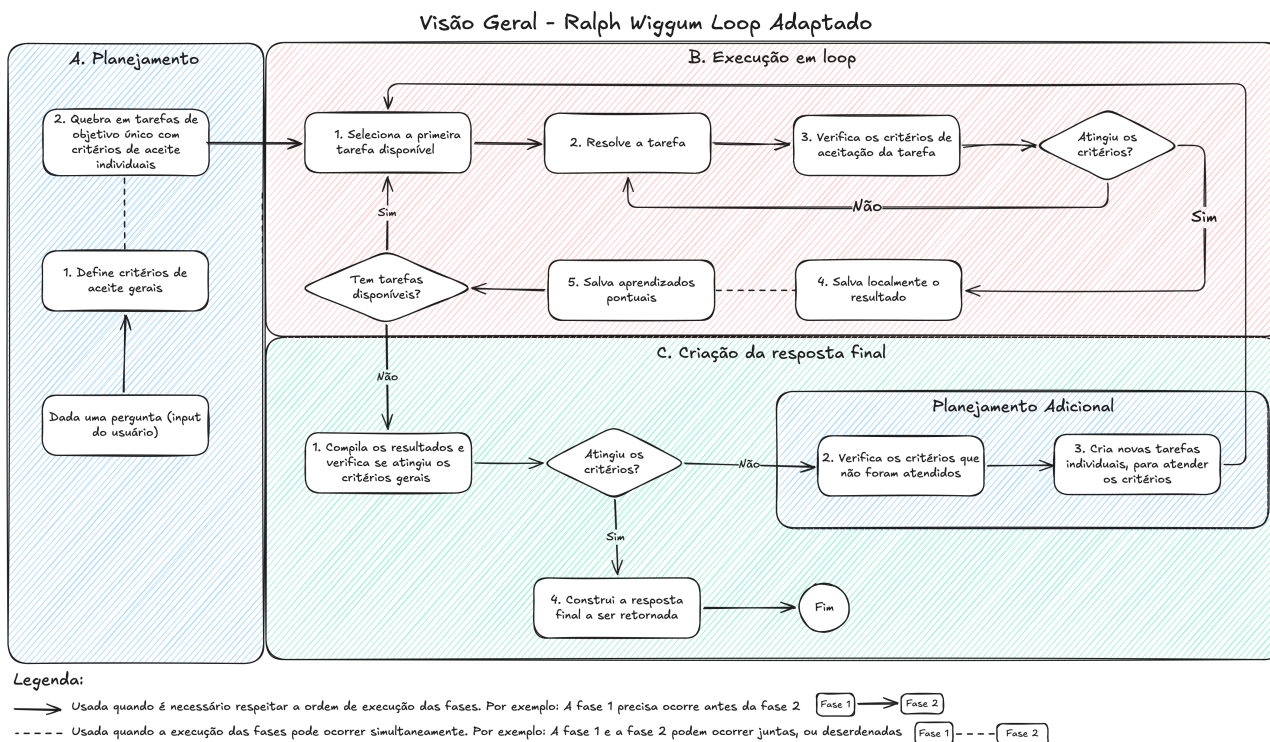


Figura 2. Visão geral do *Ralph Wiggum Loop* adaptado para resolução de perguntas em linguagem natural.

Exemplo

Considerando o *input* de referência, o modelo deveria retornar critérios como: “A pergunta deve ser respondida em Português do Brasil”; “Responda somente a alternativa, sem conter explicações adicionais”; e “Garanta que o valor encontrado ao final corresponde a uma das alternativas fornecidas”.

A.2 Quebra em tarefas de objetivo único com critérios de aceite individuais

A segunda etapa do planejamento é responsável por quebrar a pergunta do usuário em pequenas tarefas autônomas. Cada tarefa precisa ter pelo menos um critério de aceite, isto é, uma condição que define o conteúdo esperado para que a tarefa possa ser considerada concluída. A ideia central é criar tarefas que gerem insumos capazes de, em conjunto, sustentar a construção da resposta final.

A implementação utiliza um *template* fixo (Template A.2), no qual o *input* do usuário é injetado e a quebra em tarefas é solicitada. Esse *template* especifica que tarefas de objetivo único possuem uma única responsabilidade pontual, como “identificar os números primos” ou “identificar os números pares”, e nunca múltiplas responsabilidades simultâneas. Além de ter um único objetivo, cada tarefa deve ser descrita da forma mais detalhada possível, de modo a evitar desvios durante a resolução. O *template* também torna explícito que o contexto de execução de uma tarefa não é transferido para a próxima, já que somente pontos específicos sintetizados como aprendizados pontuais serão preservados. Por fim, toda tarefa deve conter pelo menos

um critério de aceite que valide se a resposta gerada contém o conteúdo necessário para encerrá-la. A relação de entrada e saída pode ser expressa como:

$$\text{extPerguntadousurio} + \text{Template A.2} \rightarrow \text{Resultado Fase A.2}$$

O resultado produzido é armazenado localmente em arquivo ou na memória do programa.

Exemplo

Considerando o *input* de referência, o modelo deveria retornar algo como:

Tarefa 1

- Tarefa: “Dadas as opções (2, 4, 6), identifique quais desses números são primos.”
- Critérios de aceite:
 - “Deve retornar a lista de números primos encontrados nas alternativas.”
 - “Deve incluir uma breve explicação da regra de identificação de números primos.”
 - “A resposta não deve tentar resolver a questão final.”
 - “A resposta não deve avaliar nenhuma outra condição.”

Tarefa 2

- Tarefa: “Dadas as opções (2, 4, 6), identifique quais desses números são pares.”

- Critérios de aceite:
 - “Deve retornar a lista de números pares encontrados nas alternativas.”
 - “Deve incluir uma breve explicação da regra de identificação de números pares.”
 - “A resposta não deve tentar resolver a questão final.”
 - “A resposta não deve avaliar nenhuma outra condição.”

Tarefa 3

- Tarefa: “Dadas as opções (2, 4, 6), identifique quais desses números são estritamente maiores que 2.”
- Critérios de aceite:
 - “Deve retornar a lista de números maiores que 2 encontrados nas alternativas.”
 - “Deve incluir uma breve justificativa matemática para a comparação estabelecida.”
 - “A resposta não deve tentar resolver a questão final.”
 - “A resposta não deve avaliar nenhuma outra condição.”

B. Execução em Loop

A fase de execução preserva a estrutura iterativa do Ralph Wiggum Loop original, mas adapta o mecanismo de conclusão de cada unidade de trabalho. Na formulação original, a conclusão de uma história de usuário é verificada por meio da execução de testes automatizados e da análise dos critérios de aceite associados ao código produzido. No fluxo adaptado, como não há código a ser testado, o modelo simplesmente resolve a tarefa selecionada e, em seguida, verifica se o resultado satisfaz os critérios de aceite individuais definidos no planejamento. Caso os critérios não sejam atingidos, o modelo repete a resolução da mesma tarefa até que sejam satisfeitos.

Ao concluir uma tarefa com sucesso, o modelo realiza dois registros distintos com finalidades diferentes: (i) persiste a resposta aprovada em armazenamento local ou na memória da aplicação, em formato suficientemente estruturado para que cada resultado possa ser recuperado individualmente, servindo de insumo para a Criação da Resposta Final; e (ii) extrai os aprendizados pontuais da execução, isto é, informações secas e objetivas descobertas ao longo das tentativas, que serão injetados no contexto da próxima tarefa como mecanismo de *Fresh Context*. É fundamental ressaltar que esses dois registros têm escopos completamente distintos: os resultados armazenados alimentam exclusivamente a fase C; os aprendizados pontuais têm papel restrito à Execução em Loop e não participam da construção da resposta final. Antes de iniciar a execução da tarefa seguinte, o contexto é reinicializado e os aprendizados acumulados são injetados como primeira entrada do novo

contexto, de forma estruturada e explícita. Esse mecanismo garante que o conhecimento relevante seja preservado sem que o ruído e as redundâncias das etapas anteriores sejam herdadas, mitigando ativamente o fenômeno de *context rot* [10].

Após cada tarefa concluída, o fluxo verifica programaticamente se ainda existem tarefas disponíveis na lista produzida pelo planejamento. Caso haja, uma nova iteração é iniciada: o contexto é reinicializado, os aprendizados acumulados são injetados e a próxima tarefa é selecionada. Caso contrário, o fluxo avança para a fase de Criação da Resposta Final, levando consigo apenas os resultados armazenados, sem os aprendizados pontuais. Essa verificação é puramente programática e não envolve chamada ao LLM.

B.1 Seleção da tarefa e resolução

Essa etapa é responsável por selecionar a próxima tarefa disponível na lista gerada pelo planejamento e submetê-la ao modelo para resolução. Ao iniciar uma nova tarefa, e somente nesse momento, o contexto é reinicializado. Os aprendizados pontuais acumulados nas tarefas anteriores são injetados no início do novo contexto, antes da descrição da tarefa atual, garantindo continuidade do conhecimento relevante sem carregar o ruído das execuções anteriores. Na primeira tarefa do ciclo, não há aprendizados acumulados, de modo que o contexto inicial contém apenas a tarefa a ser resolvida.

A implementação utiliza um *template* fixo (Template B.1) que orienta o modelo a focar exclusivamente no objetivo da tarefa atual, sem tentar antecipar ou resolver etapas posteriores. O contexto de entrada é montado a partir de três componentes: os aprendizados pontuais acumulados das tarefas anteriores (quando houver), a descrição completa da tarefa selecionada conforme definida no planejamento, e o próprio Template B.1. A relação de entrada e saída pode ser expressa como:

extAprendizadosAcumulados + Tarefa Selecionada
+ Template B.1 → Resultado Preliminar B

Exemplo

Considerando a Tarefa 1 como primeira tarefa do ciclo, o contexto não contém aprendizados acumulados. O modelo deveria retornar algo como:

“Entre as alternativas (2, 4, 6), somente o número 2 é primo. Um número primo é aquele divisível exclusivamente por 1 e por ele mesmo. O número 4 possui divisores adicionais (1, 2 e 4) e o 6 também (1, 2, 3 e 6), portanto nenhum dos dois é primo. Resultado: [2].”

B.2 Verificação dos critérios de aceite individuais

Essa etapa verifica se o Resultado Preliminar B produzido na etapa anterior atende a todos os critérios de aceite individuais associados à tarefa em execução. Trata-se de um

mecanismo de controle de qualidade interno: se todos os critérios forem atendidos, o fluxo avança para o armazenamento do resultado; caso contrário, a tarefa é reapresentada ao modelo para uma nova tentativa. É importante destacar que, ao contrário do que ocorre na transição entre tarefas distintas, o contexto *não* é reinicializado durante as tentativas de repetição de uma mesma tarefa. Essa escolha é intencional: a tentativa anterior, com os critérios não satisfeitos explicitamente sinalizados, permanece no contexto e auxilia o modelo a identificar o que deve ser corrigido na próxima execução.

A implementação utiliza um *template* fixo (Template B.2) que orienta o modelo a verificar, critério por critério, se o resultado atende ao esperado, retornando uma decisão estruturada (aprovado ou reprovado) e, em caso de reprovação, identificando quais critérios não foram satisfeitos. A relação de entrada e saída pode ser expressa como:

extResultadoPreliminarB + Critérios da Tarefa
+Template B.2 → Decisão

Exemplo

Cenário de aprovação. Avaliando o resultado da Tarefa 1 contra seus critérios de aceite:

- “Deve retornar a lista de números primos.”
Atendido: a lista [2] foi retornada.
- “Deve incluir explicação da regra de identificação de primos.”
Atendido: a explicação por divisores foi fornecida.
- “A resposta não deve tentar resolver a questão final.”
Atendido: o modelo não antecipou a resposta.
- “A resposta não deve avaliar nenhuma outra condição.”
Atendido: somente a primalidade foi avaliada.

Decisão: **Aprovado.** O fluxo avança para o armazenamento do resultado.

Cenário de reprovação e nova tentativa. Considere a Tarefa 2 (“identifique quais números são pares”). O aprendizado acumulado da Tarefa 1 já está no contexto: “O único número primo entre as alternativas é o 2”. Suponha que o modelo produza na primeira tentativa: “Todos os números 2, 4 e 6 são pares, pois são divisíveis por 2. No entanto, como apenas o 2 é primo (identificado anteriormente) e o 2 não é maior que 2, nenhuma alternativa satisfaz simultaneamente as três condições. Portanto, a resposta é D) Nenhuma das anteriores.”

Avaliando esse resultado contra os critérios da Tarefa 2:

- “Deve retornar a lista de números pares.”
Atendido: os três números foram identificados.
- “Deve incluir explicação da regra de identificação de pares.”
Atendido: a divisibilidade por 2 foi mencionada.

- “A resposta não deve tentar resolver a questão final.”
Não atendido: o modelo antecipou a resposta final.
- “A resposta não deve avaliar nenhuma outra condição.”
Não atendido: o modelo cruzou condições de outras tarefas.

Decisão: **Reprovado.** O fluxo retorna à resolução da Tarefa 2, mantendo o contexto com a tentativa reprovada e os critérios não satisfeitos identificados.

Na segunda tentativa, com o *feedback* anterior no contexto, o modelo retorna:

“Entre as alternativas (2, 4, 6), todos os três números são pares, pois cada um deles é divisível por 2 sem deixar resto ($2 \div 2 = 1$, $4 \div 2 = 2$, $6 \div 2 = 3$). Resultado: [2, 4, 6].”

Todos os critérios da Tarefa 2 são agora atendidos.
Decisão: **Aprovado.** O fluxo avança.

B.3 Armazenamento do resultado

Quando a resposta de uma tarefa é aprovada na verificação dos critérios de aceite, ela é persistida localmente. Somente essa resposta aprovada é armazenada: todo o ruído gerado pelos ciclos de tentativa anteriores, incluindo respostas reprovadas, histórico de verificações e *feedback* intermediário, não é persistido e será descartado quando o contexto for reinicializado na transição para a próxima tarefa. O armazenamento é indexado por tarefa, de modo que cada resultado possa ser recuperado individualmente pela fase de Criação da Resposta Final. Esses resultados armazenados são o único insumo factual utilizado na fase C para a construção da resposta ao usuário; os aprendizados pontuais, tratados na etapa seguinte, têm escopo restrito à Execução em Loop e não participam da criação da resposta final.

Do ponto de vista de implementação, a resposta aprovada na etapa B.2 é persistida no armazenamento local ou na memória da aplicação, indexada pelo identificador da tarefa. O formato de armazenamento é uma decisão de implementação, sendo que qualquer estrutura que permita recuperar individualmente o resultado de cada tarefa é adequada. Não há chamada adicional ao LLM nessa etapa. A relação de entrada e saída pode ser expressa como:

Resposta Aprovada → Armazenamento local

Exemplo

A Tarefa 2 exigiu duas tentativas antes de ser aprovada. Somente a segunda é armazenada. A estrutura abaixo utiliza JSON a título ilustrativo; qualquer representação equivalente é válida:

```
{ "tarefa": 2, "resultado": "Entre as alternativas (2, 4, 6), todos os três números são pares, pois cada um é divisível por 2 sem deixar
```

resto. Resultado: [2, 4, 6]."}}

A primeira tentativa reprovada não é persistida. Ao término da Execução em Loop, o armazenamento contém exclusivamente as respostas aprovadas de cada tarefa:

- { "tarefa": 1, "resultado": "O único número primo entre (2, 4, 6) é o 2. Os números 4 e 6 possuem divisores além de 1 e eles mesmos." }
- { "tarefa": 2, "resultado": "Entre (2, 4, 6), todos os três números são pares: 2, 4 e 6." }
- { "tarefa": 3, "resultado": "Os números estritamente maiores que 2 entre as alternativas são 4 e 6. O número 2 não satisfaz essa condição pois $2 > 2$ é falso." }

Esse conjunto é entregue à fase C como base exclusiva para a construção da resposta final.

B.4 Armazenamento dos aprendizados pontuais

Os aprendizados pontuais funcionam como as “anotações internas” do fluxo de *thinking*: são fatos secos e objetivos descobertos durante a execução de uma tarefa que podem ser úteis para resolver as tarefas subsequentes. Exemplos típicos incluem o resultado de um cálculo intermediário, uma fórmula identificada, uma conversão de unidade de medida, um valor recuperado após busca ou uma lista de candidatos filtrados. Em todos os casos, o registro deve conter apenas a informação em si, sem qualquer contexto de como ela foi obtida. A extração de aprendizados considera o contexto completo da tarefa, incluindo as tentativas reprovadas, pois fatos úteis podem ter emergido mesmo em execuções que não passaram na verificação de critérios. Os aprendizados pontuais têm escopo estritamente restrito à Execução em Loop: são injetados no início do contexto de cada nova tarefa para subsidiar a resolução, mas não são incluídos, em nenhuma hipótese, na fase de Criação da Resposta Final.

A implementação utiliza um *template* fixo (Template B.4) aplicado sobre o contexto completo da tarefa concluída. Esse *template* instrui o modelo a identificar fatos concretos que possam ser úteis nas próximas tarefas, registrá-los em formato de lista seca sem narrativa ou contexto, e descartar todo o restante. Os aprendizados extraídos são adicionados à memória acumulada, que será injetada no início do contexto da próxima tarefa. A relação de entrada e saída pode ser expressa como:

extContextoCompleto da Tarefa + Template B.4
→ Aprendizados Pontuais → Memória Acumulada

Exemplo

Após a conclusão das três tarefas, a memória acumulada conteria algo como:

“Primos entre (2, 4, 6): [2]”

“Pares entre (2, 4, 6): [2, 4, 6]”

“Maiores que 2 entre (2, 4, 6): [4, 6]”

Esse bloco compacto seria injetado no início do contexto de uma eventual Tarefa 4 e não é utilizado na construção da resposta final.

Como contraponto, o trecho a seguir ilustra o formato **inválido** de aprendizado, por ser verboso e conter contexto desnecessário:

Inválido: “Durante a execução da Tarefa 2, após uma tentativa reprovada por ter antecipado a resposta final, foi identificado que todos os números 2, 4 e 6 são pares, verificando a divisibilidade de cada um por 2.”

O aprendizado pontual válido extrai apenas o fato: “Pares entre (2, 4, 6): [2, 4, 6]”.

A distinção entre resultado armazenado (etapa B.3) e aprendizado pontual (etapa B.4) é estrutural e intencional. O resultado é a resposta completa e verificada da tarefa, destinada à síntese da fase C. O aprendizado é um fragmento seco de fato, destinado exclusivamente a subsidiar a execução das tarefas seguintes dentro do loop.

C. Criação da Resposta Final

A terceira fase constitui uma adição exclusiva do fluxo adaptado, sem correspondência direta na formulação original. No *Ralph Wiggum Loop* concebido para engenharia de software, o produto final é o próprio código implementado e versionado ao longo das histórias; não há, portanto, necessidade de uma etapa de síntese do conteúdo e geração de resposta. No fluxo adaptado, os resultados das tarefas individuais são fragmentos de raciocínio que precisam ser compilados em uma resposta coesa e completa.

Após o encerramento do loop, o modelo compila os resultados armazenados e verifica se as informações produzidas são suficientes para satisfazer todos os critérios de aceite gerais definidos no planejamento. Se os critérios forem integralmente atendidos, o modelo constrói a resposta final e a retorna ao usuário. Caso contrário, o sistema infere que o planejamento inicial foi insuficiente para cobrir todos os aspectos da pergunta e inicia uma fase de *Planejamento Adicional*: o modelo identifica os critérios ainda não atendidos, cria novas tarefas individuais para supri-los e retorna à fase de Execução em Loop. Esse mecanismo garante que lacunas de planejamento sejam identificadas e corrigidas antes que a resposta final seja construída, preservando a completude e a coerência da saída.

C.1 Compilação dos resultados e verificação dos critérios gerais

Essa etapa reúne todos os resultados armazenados durante a Execução em Loop e verifica se o conjunto de informações produzido é suficiente para satisfazer cada

um dos critérios de aceite gerais definidos na etapa A.1. A distinção central em relação à etapa B.2 é o escopo da verificação: enquanto lá a avaliação é local, com cada tarefa verificada contra seus próprios critérios individuais, aqui a avaliação é global, e o conjunto completo de resultados é considerado para determinar se é possível construir uma resposta final completa e correta.

A implementação utiliza um *template* fixo (Template C.1) que instrui o modelo a listar os resultados de cada tarefa, avaliar critério geral por critério geral se os resultados em conjunto permitem satisfazê-los, e retornar uma decisão estruturada com identificação dos critérios não atendidos, quando for o caso. A relação de entrada e saída pode ser expressa como:

extResultadosArmazenados + Critérios Gerais (A.1)
+Template C.1 $\rightarrow$ Decisão

Exemplo

Cenário de aprovação. Com os três resultados disponíveis ao término do loop (T1: o único primo é [2]; T2: todos os números são pares [2, 4, 6]; T3: os maiores que 2 são [4, 6]), a avaliação contra os critérios gerais de A.1 produz:

- “A pergunta deve ser respondida em Português do Brasil.”
Atendível: todos os resultados estão em português.
- “Responda somente a alternativa, sem explicações adicionais.”
Atendível: os dados disponíveis permitem identificar a alternativa correta.
- “O valor encontrado deve corresponder a uma das alternativas.”
Atendível: $\text{primo} \cap \text{par} \cap \text{maior que } 2 = \emptyset$, o que corresponde à alternativa D.

Decisão: **Aprovado.** O fluxo avança para a construção da resposta final.

Cenário de reprovação. Considere uma variação em que os critérios gerais incluam: “A resposta deve ser sustentada por uma verificação explícita de qual conjunto de alternativas satisfaz simultaneamente todas as condições enunciadas.” Os três resultados cobrem cada condição individualmente, mas nenhuma tarefa realizou a análise de interseção:

- “Verificação explícita da interseção dos conjuntos.”
Não atendido: nenhuma tarefa realizou a verificação combinada das três condições.

Decisão: **Reprovado.** O fluxo avança para o Planejamento Adicional.

C.2 Planejamento Adicional

Quando os critérios gerais não são atendidos, o Planejamento Adicional funciona como uma extensão da fase A.2: identifica as lacunas específicas nos resultados disponíveis e cria novas tarefas individuais para supri-las. Diferentemente do planejamento inicial, que parte da pergunta do usuário sem qualquer contexto de execução, o Planejamento Adicional tem acesso ao conjunto de resultados já produzidos e ao mapeamento preciso dos critérios que ainda não foram satisfeitos. Essa informação torna as novas tarefas mais pontuais e direcionadas, evitando retrabalho sobre o que já foi verificado. As novas tarefas são adicionadas à fila de execução e o fluxo retorna à etapa B.1, agora com o contexto dos aprendizados acumulados de todas as tarefas anteriores disponível.

A implementação utiliza um *template* fixo (Template C.2) que orienta o modelo a identificar os critérios gerais não atendidos, raciocinar sobre que tipo de informação adicional seria necessária para satisfazê-los, e formular novas tarefas de objetivo único com critérios de aceite individuais, seguindo o mesmo padrão estabelecido na fase A.2. A relação de entrada e saída pode ser expressa como:

extCritriosNoAtendidos + Resultados Armazenados
+Template C.2 $\rightarrow$ Novas Tarefas

Exemplo

A partir da reprovação identificada no cenário anterior, o Planejamento Adicional produziria algo como:

Tarefa 4

- Tarefa: “Dadas as listas já identificadas — primos: [2]; pares: [2, 4, 6]; maiores que 2: [4, 6] —, determine quais números aparecem simultaneamente nas três listas.”
- Critérios de aceite:
 - “Deve retornar o resultado da interseção das três listas, podendo ser um conjunto vazio.”
 - “Deve apresentar justificativa explicitando a presença ou ausência de cada candidato em cada lista.”
 - “A resposta não deve construir a resposta final ao usuário.”

Com os aprendizados de T1, T2 e T3 injetados no contexto, o modelo executa a Tarefa 4 e retorna algo como:

“Verificando a interseção: primos [2] $\cap$ pares [2, 4, 6] $\cap$ maiores que 2 [4, 6]. O número 2 está em {primos} e {pares}, mas não em {maiores que 2} (pois $2 > 2$ é falso). Os números 4 e 6 estão em {pares} e {maiores que 2}, mas não em {primos}. Resultado da interseção: {} (conjunto vazio).”

Após a aprovação da Tarefa 4, o fluxo retorna a C.1. Com quatro resultados disponíveis, todos os critérios

gerais são atendidos e o fluxo avança para a construção da resposta final.

O mecanismo de Planejamento Adicional pode ocorrer mais de uma vez caso as novas tarefas criadas ainda não sejam suficientes para satisfazer todos os critérios gerais. A cada retorno a C.1, o conjunto de resultados disponíveis é maior, convergindo progressivamente para a completude.

C.3 Construção da resposta final

Com todos os critérios gerais satisfeitos, o modelo compõe a resposta final ao usuário. Essa etapa não realiza novo raciocínio analítico sobre o problema: seu objetivo é exclusivamente organizar e apresentar os resultados já produzidos no formato e idioma definidos pelos critérios gerais (fase A.1). A distinção entre esta etapa e as anteriores é deliberada: ao separar o raciocínio analítico, realizado de forma incremental e verificada durante a Execução em Loop, da síntese da resposta final, reduz-se o risco de que a fase de apresentação introduza inferências não verificadas ou contradiga os resultados produzidos.

A implementação utiliza um *template* fixo (Template C.3) que instrui o modelo a utilizar exclusivamente os resultados armazenados como base factual, respeitar o formato e o idioma definidos pelos critérios gerais, e não introduzir raciocínio ou informações novas que não estejam amparadas pelos resultados das tarefas. A relação de entrada e saída pode ser expressa como:

extResultadosCompilados + Critérios Gerais (A.1)
+ Template C.3 → Resposta Final

Exemplo

Em ambos os cenários (aprovação direta com três tarefas ou com Planejamento Adicional em quatro tarefas), os resultados indicam que primo $\cap$ par $\cap$ maior que 2 = $\emptyset$. Respeitando os critérios gerais — resposta em Português do Brasil, somente a alternativa, sem explicações adicionais, correspondente a uma das alternativas fornecidas —, a resposta final produzida é:

D) Nenhuma das anteriores

Essa resposta é então retornada ao usuário, encerrando o fluxo.

IV. Cronograma do projeto

V. Referências

Referências

- [1] Y. Lee, R. Nair, Q. Zhang, K. Lee, O. Khattab e C. Finn, “Meta-Harness: End-to-End Optimization of Model Harnesses,” *arXiv preprint arXiv:2603.28052*, 2026.
- [2] J. Wei et al., “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models,” *arXiv preprint arXiv:2201.11903*, 2022.
- [3] S. Yao et al., “Tree of Thoughts: Deliberate Problem Solving with Large Language Models,” *arXiv preprint arXiv:2305.10601*, 2023.
- [4] D. Zhou et al., “Least-to-most prompting enables complex reasoning in large language models,” *arXiv preprint arXiv:2205.10625*, 2022.
- [5] S. Yao et al., “ReAct: Synergizing Reasoning and Acting in Language Models,” *arXiv preprint arXiv:2210.03629*, 2022.
- [6] N. Shinn, F. Cassano, B. Labash, A. Gopinath, K. Narasimhan e S. Yao, “Reflexion: Language Agents with Verbal Reinforcement Learning,” *arXiv preprint arXiv:2303.11366*, 2023.
- [7] X. Chen, M. Lin, N. Schärli e D. Zhou, “Teaching Large Language Models to Self-Debug,” *arXiv preprint arXiv:2304.05128*, 2023.
- [8] A. Madaan et al., “Self-Refine: Iterative Refinement with Self-Feedback,” *arXiv preprint arXiv:2303.17651*, 2023.
- [9] N. F. Liu et al., “Lost in the Middle: How Language Models Use Long Contexts,” *Transactions of the Association for Computational Linguistics*, v. 12, pp. 277–294, 2024.
- [10] K. Hong, A. Troynikov e J. Huber, “Context Rot: How Increasing Input Tokens Impacts LLM Performance,” Chroma, rel. técn., jul. de 2025. endereço: <https://trychroma.com/research/context-rot>
- [11] G. Ghuntley, *Ralph Loop: A Structured Orchestration Pattern for LLM Inference*, Online. Disponível em: <https://ghuntley.com/specs/ralph/>, Acesso em: abr. 2026, 2025.
- [12] Z. Wang et al., “Beyond the Strongest LLM: Multi-Turn Multi-Agent Orchestration vs. Single LLMs on Benchmarks,” *arXiv preprint arXiv:2509.23537*, 2025.
- [13] D. Hendrycks et al., “Measuring Massive Multitask Language Understanding,” em *International Conference on Learning Representations*, 2021.
- [14] D. Rein et al., “GPQA: A Graduate-Level Google-Proof Q&A Benchmark,” *arXiv preprint arXiv:2311.12022*, 2023.
- [15] T. B. Brown et al., “Language Models are Few-Shot Learners,” em *Advances in Neural Information Processing Systems*, 2020.
- [16] A. Vaswani et al., “Attention is All You Need,” em *Advances in Neural Information Processing Systems*, vol. 30, 2017.
- [17] W. X. Zhao et al., “A Survey of Large Language Models,” *arXiv preprint arXiv:2303.18223*, 2023.

Tabela I. Cronograma das atividades do projeto.

Atividade	fev	mar	abr	mai	jun	jul	ago	set	out	nov
Revisão bibliográfica e refinamento da proposta	X	X	X	X						
Definição da arquitetura		X	X	X	X					
Desenvolvimento do Paper		X	X	X	X	X	X	X	X	X
Implementação prática do código				X	X	X	X	X	X	
Análise final									X	X

- [18] M. Zaharia et al., *The Shift from Models to Compound AI Systems*, Berkeley Artificial Intelligence Research Blog. Disponível em: <https://bair.berkeley.edu/blog/2024/02/18/compound-ai-systems/>, Acesso em: mai. 2026, 2024.
- [19] Anthropic, *Claude Code*, <https://www.anthropic.com/claude-code>, 2026.
- [20] M. Chen et al., “Evaluating Large Language Models Trained on Code,” *arXiv preprint arXiv:2107.03374*, 2021.
- [21] *OpenCLI*, <https://opencli.ai/>, 2026.
- [22] T. Khot et al., “Decomposed prompting: A modular approach for solving complex tasks,” *arXiv preprint arXiv:2210.02406*, 2022.
- [23] L. Wang et al., “Plan-and-solve prompting: Improving zero-shot chain-of-thought reasoning by large language models,” *arXiv preprint arXiv:2305.04091*, 2023.
- [24] C. McComb et al., “Supervising Ralph Wiggum: Exploring a Metacognitive Co-Regulation Agentic AI Loop for Engineering Design,” *arXiv preprint arXiv:2603.24768*, 2026.